Introduction to ROS:
Group Project: Autonomous Driving

Gaojian Xie, Haowen Shi, Jia Cao, Lingheng Kong, Zhuoya Zhang

Supervisor: Prof. Dr. Markus Ryll, Ojas Mandlekar

July 2025

1 Introduction

This project was carried out as part of the Introduction to ROS course at TUM. The main objective is to develop a vehicle capable of autonomously navigating a predefined urban track while interacting with dynamic elements in Unity. The system is implemented using a ROS-based framework integrated with a Unity simulation environment. Core components of the project include a perception pipeline, path and trajectory planning modules, a vehicle control algorithm, and a decision-making logic. In this documentation, we provide a overview of the system architecture, implemented ROS nodes and packages, key design decisions, challenges encountered during development, and the resulting performance. Supporting materials such as ROS graphs, figures, and plots are also included to illustrate our approach and outcomes.

2 Project Goals

The project aims to develop an autonomous driving system capable of navigating an urban environment within a Unity simulated environment. Key objectives include:

- Designing a perception module to extract geometric and semantic information from camera data.
- Implementing a path and trajectory planner that ensures safe and efficient motion through a predefined track.
- Integrating a control module that translates planned trajectories into executable commands.
- Establishing robust communication between ROS nodes and the Unity-based simulator to enable real-time feedback and sensor emulation.
- Developing a decision-making mechanism to manage complex interactions such as traffic light handling and emergency braking.

3 Standard Package Overview

3.1 ROS Noetic depth_image_proc

The `depth_image_proc` package in ROS Noetic is used to convert depth images (`sensor_msgs/Image`) into 3D point clouds (`sensor_msgs/PointCloud2`). Specifically, the `point_cloud_xyz` nodelet processes synchronized depth and camera info topics to generate dense point clouds that represent the 3D structure of the environment. This step is essential for feeding spatial data into mapping or planning modules like OctoMap.

3.2 ROS Noetic octomap_server

The OctoMap framework in ROS Noetic provides an efficient solution for representing three-dimensional environments using a probabilistic occupancy mapping approach. It constructs 3D octree-based voxel grids from depth sensor inputs, such as stereo camera or depth camera data, allowing scalable and memory-efficient storage through adaptive resolution. Within the ROS ecosystem, the `octomap_server` node facilitates the real-time generation and publication of both full 3D occupancy maps and their 2D projections, which can be directly integrated into navigation, path planning, and obstacle avoidance modules for autonomous vehicles or mobile robots.

3.3 OpenCV

OpenCV (Open Source Computer Vision Library) is an open-source library widely used in the fields of computer vision and image processing, including tasks such as object detection, image analysis, and feature extraction.

In our project, OpenCV plays a crucial role in the following aspects:

- **Image Format Conversion**: Used in combination with `cv_bridge` to convert ROS image messages into `cv::Mat` format for further processing.
- **Color Detection and Segmentation**: Implements traffic light color extraction (red and yellow) using `cv::inRange` in the HSV color space.
- **Geometric Operations**: Utilized for tasks such as detecting the number of traffic lights (`findContours`), color space conversion (`cvtColor`), and calculating the red pixel ratio (`countNonZero`).
- **Image Cropping and ROI Extraction**: Extracts regions of interest (ROIs) in the RGB image corresponding to detected traffic lights for state classification.

Overall, OpenCV provides core functionality for visual recognition and color analysis in the system, enabling accurate and efficient traffic light detection.

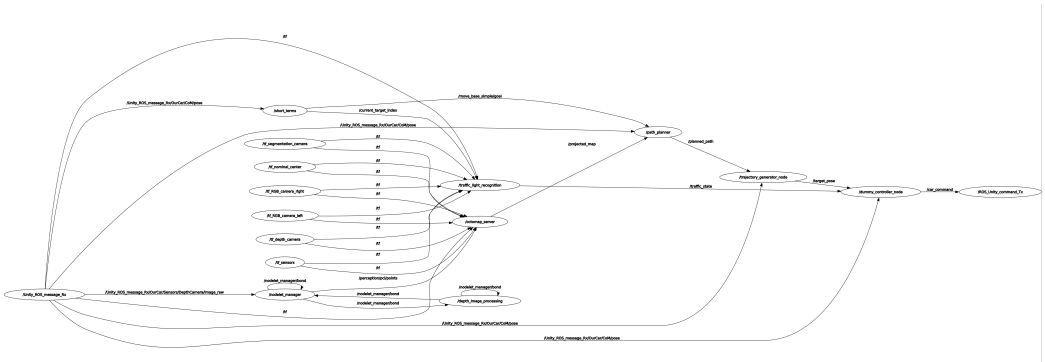

Figure 1: The rqt graph of the whole project

4 Implementation

4.1 Unity Environment and ROS-Simulation-Bridge

Unity serves as the core simulation platform for autonomous driving, replicating a realistic urban environment with roads, intersections, traffic lights, and dynamic agents like NPC vehicles. It provides key sensor data—such as depth and RGB camera images, semantic segmentation, and INS readings—and updates the vehicle's position and surroundings in real time.

Unity communicates with ROS through a TCP-based simulation bridge, which offers access to sensor topics and services. This allows ROS nodes to receive data and send control commands, enabling realistic vehicle behavior including steering, acceleration, and braking.

The simulation also supports configurable features like camera settings, spawn positions, optional events, and ground removal in depth images. This setup creates an efficient and flexible platform for developing and testing perception, planning, and control algorithms in a safe, high-fidelity environment.

4.2 Perception pipeline

The perception pipeline begins with the depth camera integrated in Unity, which continuously generates depth images and publishes them to the topic `/OurCar/Sensors/DepthCamera/image_raw` as messages of type `sensor_msgs/Image`. This simulated depth data serves as the foundation for 3D environmental perception.

To convert the raw depth images into spatially meaningful data, we utilize the `depth_image_proc` package—specifically, the `depth_image_proc/point_cloud_xyz` nodelet. This nodelet transforms the

depth images into a 3D point cloud and publishes the result to `/perception/pcl/points` using the `sensor_msgs/PointCloud2` message type. The conversion process is configured via a launch file, where camera topics are properly remapped and the nodelet is loaded into a manager for efficient processing.

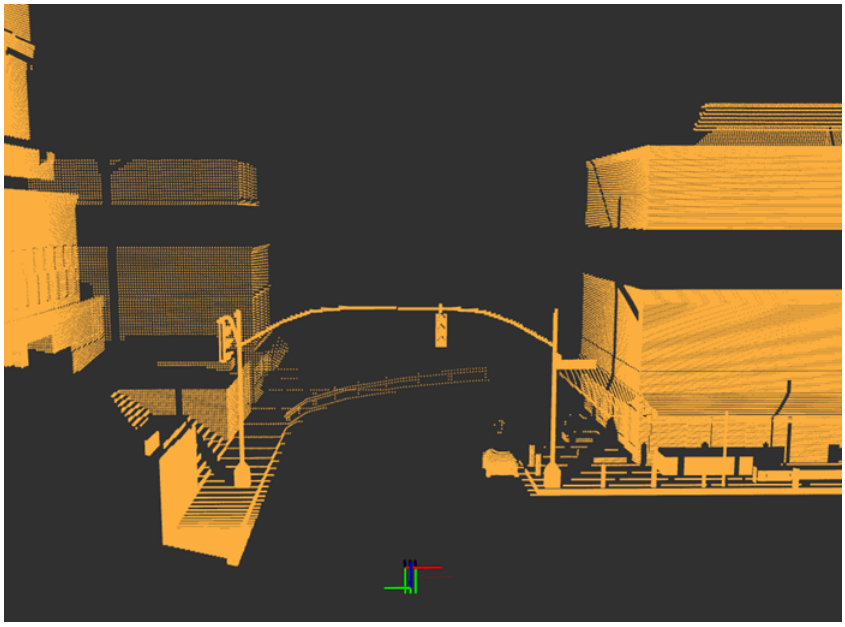

Figure 2: Point cloud

The point cloud is then fed into the `octomap_server`, which constructs a 3D OctoMap representation of the environment and publishes it on `/octomap_binary` as a message of type `octomap_msgs/Octomap`. In addition to the full 3D map, the `octomap_server` also projects the map onto a 2D occupancy grid, published to `/projected_map` using the `nav_msgs/OccupancyGrid` message type. This 2D map serves as a crucial input for the A* path planning module.

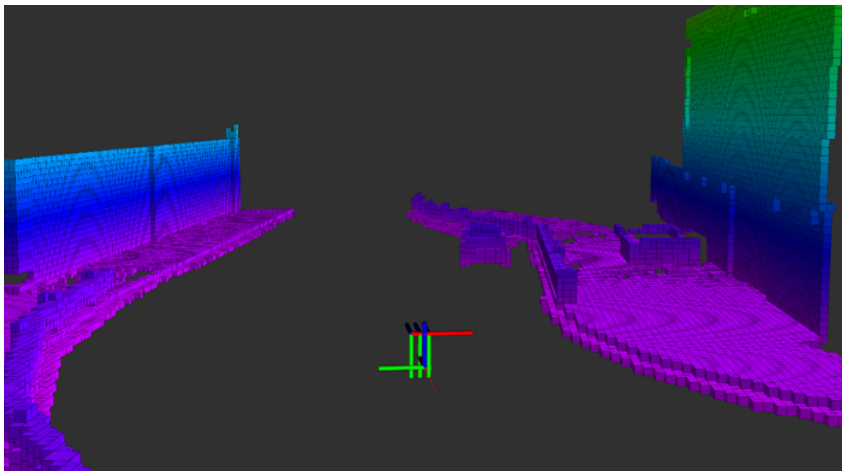

Figure 3: Octomap

To ensure spatial information is captured, parameters such as `occupancy_min_z`, `occupancy_max_z`, and `project_2d_map` are configured in the launch file. These settings help to filter out irrelevant structures like the ground and constrain the projected map to navigable space. This pipeline effectively bridges the gap between simulated visual perception and actionable spatial understanding for autonomous navigation.

4.3 Path Planner

4.3.1 Overview

The `path_planner` package contains three main nodes: `path_planner_node`, `pose_recorder`, and `short_term_planner`. To generate the reference route, we first manually drive the vehicle along the desired path within the Unity simulation environment. During this process, the `pose_recorder` node automatically records the vehicle's pose at fixed distance intervals and saves the data to a csv file.

During autonomous driving, the recorded poses are published as short-term goals by `short_term_planner` node. The `path_planner_node`, which is based on an improved A* algorithm derived from code provided by ChatGPT, subscribes to 2D occupancy grids and the vehicle's current state, and computes a global path from the current position to the given short-term goal. The result is published as a `nav_msgs::Path` message.

4.3.2 Short-Term Goal Publishing

The `short_term_planner` node is responsible for sequentially publishing intermediate goals during autonomous navigation.

It subscribes to the vehicle's current pose (`/Unity_ROS_message_Rx/OurCar/CoM/pose`) and waits for the system readiness signal via `/path_planner_ready`. Once ready, it loads a series of target waypoints from a file (`car_positions.txt`) located in the package directory, where each line corresponds to an (x, y) coordinate. As the vehicle moves, the node monitors its distance to the current goal. When within a threshold (default: 4.5 meters), it:

- Publishes the next goal to `/move_base_simple/goal`
- Updates and publishes the goal index via `/current_target_index`

The process continues until all waypoints are reached. Internally, the node maintains:

- A list of waypoints: `std::vector<std::pair<double, double» target_positions_`
- A distance threshold: `double distance_threshold = 4.5`
- Flags to track pose reception and goal update: `pose_received_`, `goal_sent_`

4.3.3 Subscribed Topics

- `/projected_map` (`nav_msgs::OccupancyGrid`): 2D map from OctoMap projection.
- `/Unity_ROS_message_Rx/OurCar/CoM/pose` (`geometry_msgs::PoseStamped`): current pose.
- `/move_base_simple/goal` (`geometry_msgs::PoseStamped`): goal pose set via RViz.

4.3.4 Published Topics

- `/planned_path` (`nav_msgs::Path`): final path in world coordinates.
- `/path_planner_ready` (`std_msgs::Bool`): signals readiness after map inflation.

4.3.5 Core Functionalities

1. **Map Handling**
   - Subscribes to 2D map, extracts resolution, size, and origin.
   - Inflates obstacles with configurable radius.
2. **Pose Management**
   - Transforms world pose to grid indices.
   - Handles invalid start points with fallback.
   - Suppresses planning if already near goal.
3. **A* Path Planning**

- Custom A* using `std::priority_queue`.
- Supports 8-connected grids with diagonals.
- Ignores minor frontal obstacles via forward ignore zone.

4. **Path Output**
   - Converts path from grid to world coordinates.
   - Publishes path for visualization or execution.

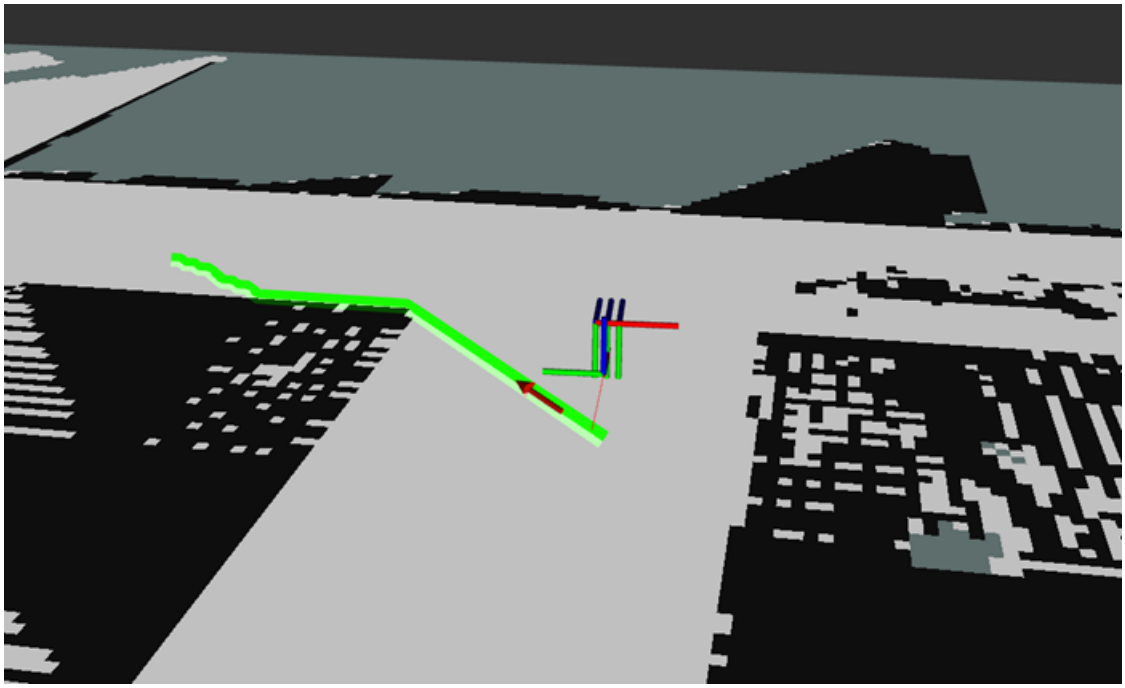

Figure 4: Generated path upon the 2D occupancy map

4.4 Trajectory generation

This node belongs to the `trajectory_generator` package. Its primary role is to serve as a bridge between path planning and the vehicle's controller. Specifically, it converts a planned geometric path (`/planned_path`) into a time-parameterized trajectory, providing velocity and orientation commands suitable for execution by the controller.

4.4.1 Subscribed Topics

- `planned_path` (`nav_msgs/Path`): the discrete path points published by the upstream path planner
- `/Unity_ROS_message_Rx/OurCar/CoM/pose` (`geometry_msgs/PoseStamped`): the current vehicle pose for nearest-point matching and closed-loop progression.

4.4.2 Timer Configuration

Iterates through trajectory points at a fixed frequency (default 10 Hz), based on the time_interval parameter.

4.4.3 Published Topics

- `velocity` (`geometry_msgs/TwistStamped`): feedforward linear and angular velocity commands, in the local (`base_link`) frame
- `target_pose` (`geometry_msgs/PoseStamped`): reference pose (x, y, yaw) for the controller

4.4.4 Trajectory Generation Basic Logic

1. **Path Preprocessing**
   - Extracts (x, y) and yaw from waypoints given by path planner
   - Computes distances ds between consecutive waypoints
2. **Curvature and Speed Limit Calculation**
   - Uses a 3-point method to compute curvature $k[i]$ at each point
   - Applies lateral acceleration limit to compute:

$$v_{\text{limit}}[i] = \min\left(v_{\max}, \sqrt{\frac{a_{\text{lat,max}}}{k[i]}}\right)$$

3. **Velocity Profiling**
   - Applies forward and backward passes to enforce acceleration limit $a_{\max}$
   - Ensures smooth and continuous velocity profile $v[i]$
4. **Timestamp Assignment**
   - Each trajectory point is assigned a timestamp $t[i]$, satisfying:

$$t[i] = t[i-1] + \frac{ds[i-1]}{v[i]}$$

   - These timestamps allow time-based interpolation and indexing
5. **Timer-based Publishing Loop**
   - A timer runs at frequency 1/time_interval
   - At each tick:
     - Finds the closest path point (i_0) to the current pose
     - Advances by a predefined arc length (lookahead_dist) to get target index j
   - Publishes:
     - `/velocity`: `linear.x =` $v[j]$, `angular.z =` $v[j] \cdot k[j]$
     - `/target_pose`: position $= (x[j],\ y[j])$, orientation $= yaw[j]$
6. **Endpoint Handling**
   - When the vehicle is within `hold_dist` of the final path point:
     - Stops publishing new commands

4.4.5 Technical Highlights

- Closest-point matching and lookahead improve tracking stability:
  - Each cycle, find index i_0 of the path point closest to the current pose and ahead of the car to prevent trying to go backward
  - Lookahead mechanism ensures the trajectory starts ahead of the car
  - Together, they ensure the trajectory always starts from a preferable position, increasing robustness, smooth tracking, and avoiding oscillation
- Three-point curvature estimation could more or less improve corner behavior
- Double-pass acceleration-limited velocity profiling ensures smooth motion
- Robust endpoint logic: hold $\rightarrow$ coast $\rightarrow$ stop transition (in case of no more targets)

4.4.6 Remarks

In practice, fine-tuning kinematic or dynamic constraints was not found to be critical, as the dummy controller only utilizes throttle, steering, and brake within predefined command ranges. The trajectory planner automatically produced reasonable velocity and acceleration profiles, as confirmed through simulation.

To simplify implementation, we chose to allow wider parameter ranges, which resulted in stable and smooth performance. One key adjustment was lowering `v_max` to ensure the vehicle had sufficient time to stop for red lights or NPC vehicles.

Additional parameters such as `lookahead_dist`, `hold_dist`, and `stop_after_dist` were introduced based on iterative testing. The lookahead mechanism prevents the vehicle from turning back when it overshoots a waypoint due to numerical or control errors. The holding mechanism ensures smooth passage through target points, compensating for reduced lookahead as the vehicle approaches the target. The `stop_after_dist` handles cases where no new target is available after reaching the previous one.

Higher-order interpolation methods (e.g., cubic splines) were deliberately avoided, as the path planner already provides sufficiently dense waypoints, and the resulting motion was smooth enough while keeping computation efficient.

4.5 Traffic Light and NPC vehicle Recognition

4.5.1 Purpose and Role

This node is responsible for detecting and classifying traffic lights in a simulated driving environment using input from multiple camera sensors (semantic, depth, and RGB) in a ROS-based Unity simulation.

4.5.2 Topic Subscriptions

The node subscribes to the following topics:

- **Semantic Camera**
  - `image_raw`: `/Unity_ROS_message_Rx/OurCar/Sensors/SemanticCamera/image_raw`
  - `camera_info`: `/Unity_ROS_message_Rx/OurCar/Sensors/SemanticCamera/camera_info`
- **Depth Camera**
  - `image_raw`: `/Unity_ROS_message_Rx/OurCar/Sensors/DepthCamera/image_raw`
  - `camera_info`: `/Unity_ROS_message_Rx/OurCar/Sensors/DepthCamera/camera_info`
- **RGB Camera**
  - `image_raw`: `/Unity_ROS_message_Rx/OurCar/Sensors/RGBCameraLeft/image_raw`
  - `camera_info`: `/Unity_ROS_message_Rx/OurCar/Sensors/RGBCameraLeft/camera_info`
- **Target waypoint index**
  - `/current_target_index`: helps contextually manage special behavior at traffic light locations

4.5.3 Published Topics

- `/traffic_light/image` – Cropped RGB ROI around the traffic light.
- `/traffic_state` – Traffic light state as a string ("RED" or "UNKNOWN").

4.5.4 Self-defined Message Type

To represent the detected traffic light status in a structured and informative way, we defined a custom message type named `TrafficLightState.msg`. This message contains the following fields:

- `string state`: Indicates the current traffic light state, with possible values such as "RED" or "UNKNOWN".

- `float32 confidence`: Represents the detection confidence level, ranging from 0 (no confidence) to 1 (high confidence).
- `float32 countdown`: An estimated countdown in seconds until the traffic light changes, allowing downstream modules to anticipate transitions.

This message is used to publish traffic light recognition results from the perception module and can be subscribed to by planning or decision-making nodes for behavior control.

4.5.5 Core Processing Logic

1. **Semantic Image Analysis**:
   - Detects yellow-colored traffic lights using HSV filtering.
   - Prioritizes the one closest to the center of the image (or horizontally closest in certain index ranges).
2. **Depth Retrieval**:
   - Extracts the depth value at the traffic light pixel.
   - If unavailable, searches nearby pixels or falls back to a fake depth estimation based on yellow area size and vehicle yaw.
3. **Coordinate Transformation**:
   - Converts the 2D traffic light pixel and depth into a 3D point in camera space.
   - Transforms the point into the RGB camera frame using TF lookup.
4. **RGB ROI Extraction**:
   - Projects the 3D point into the RGB image.
   - Extracts a small rectangular ROI around the projected point.
5. **State Classification**:
   - Converts the ROI to HSV and computes red pixel ratio.
   - If ratio $> 1.5\%$, the light is marked RED, otherwise UNKNOWN.
6. **Failsafe**:
   - If the semantic image contains a large red area directly ahead (e.g., red barrier), it publishes "RED" regardless of traffic light detection.

4.5.6 Key Parameters

- `process_interval_` = 3: Limits processing frequency to every 3 frames.
- `t_x_`, `t_y_`, `t_z_`: Calibration offsets used when fallback depth is applied.
- HSV color thresholds for yellow and red are fine-tuned for Unity simulation lighting.
- Dynamic ROI offset logic based on `current_index` to adapt to viewpoint shifts at intersections.

4.5.7 Design Highlights

- Combines semantic segmentation (for object localization) and RGB data (for visual classification).
- Handles missing or invalid depth robustly with multiple fallback strategies.
- Integrates TF transformation and camera projection math for spatial consistency.
- Publishes cropped traffic light images for easy visualization/debugging.
- Designed to be adaptive based on route position (`current_index`) to avoid misclassification.

4.5.8 Conclusion

This node is a self-contained, robust perception module for traffic light recognition, effectively merging camera data, geometry, and color analysis. It is designed with fault tolerance, modularity, and interpretability in mind, which is critical for real-world robotics or simulation-based autonomous driving tasks.

4.6 Controller and Decision Making

The dummy_controller_node is a node designed to perform basic trajectory tracking using a PID-based control strategy, which is implemented within the dummy_controller package. Its core function is to compute appropriate throttle, brake, and steering commands to guide the ego vehicle toward a specified target pose while responding to dynamic constraints such as current velocity and traffic light state.

To achieve this, the node subscribes to several key ROS topics: the vehicle's current pose (`/Unity_ROS_message_Rx/OurCar/CoM/pose`), twist (`/Unity_ROS_message_Rx/OurCar/CoM/twist`), target pose (`/target_pose`), desired velocity (`/velocity`), and traffic light status (`/traffic_state`).

```
pose_sub_ = nh.subscribe("/Unity_ROS_message_Rx/OurCar/CoM/pose", 1, &DummyPIDController::
    poseCallback, this);
twist_sub_ = nh.subscribe("/Unity_ROS_message_Rx/OurCar/CoM/twist", 1, &DummyPIDController::
    twistCallback, this);
target_sub_ = nh.subscribe("/target_pose", 1, &DummyPIDController::targetCallback, this);
velocity_sub_ = nh.subscribe("/velocity", 1, &DummyPIDController::velocityCallback, this);
traffic_sub_ = nh.subscribe("/traffic_state", 1, &DummyPIDController::trafficCallback, this);

cmd_pub_ = nh.advertise<simulation::VehicleControl>("car_command", 1);
```

Listing 1: ROS Subscribers and Publisher Initialization

Upon receiving sufficient input data, the node executes a control loop at 10 Hz, which fits the frequency of the trajectory_generater node and avoids delays caused by time. If the traffic light is red, the vehicle halts by applying full braking and zero throttle, resetting all PID integrals and derivatives to avoid unwanted control accumulation.

```
if (traffic_state_ == "RED") {
    speed_integral_ = 0.0;
    prev_speed_error_ = 0.0;
    steer_integral_ = 0.0;
    prev_steer_error_ = 0.0;

    cmd.Throttle = 0.0;
    cmd.Brake = 1.0; // maximum brake
    cmd.Steering = 0.0;
    prev_speed_error_ = -500.0;
}
```

Listing 2: Handling RED Traffic State

The controller begins by ensuring the target point lies ahead of the vehicle to prevent unrealistic or unstable behaviors such as moving in reverse. It does so by computing the dot product between the unit heading vector—derived from the vehicle's current yaw angle—and the vector pointing from the vehicle's current position to the target position. A negative result indicates the target lies behind the vehicle, in which case the controller immediately commands maximum braking and no throttle or steering, effectively halting the car until a valid forward-facing goal is received.

```
double dx = target_pose_.pose.position.x - current_pose_.pose.position.x;
double dy = target_pose_.pose.position.y - current_pose_.pose.position.y;
double current_yaw = tf::getYaw(current_pose_.pose.orientation);
double heading_x = std::cos(current_yaw);
double heading_y = std::sin(current_yaw);
double dot = dx * heading_x + dy * heading_y;

if (dot < 0) {
```

```
    ROS_WARN_STREAM("The target point is behind the car, ignore it");
    cmd.Throttle = 0.0;
    cmd.Brake = 1.0;
    cmd.Steering = 0.0;
    cmd_pub_.publish(cmd);
    ros::spinOnce();
    loop_rate.sleep();
    continue;
}
```

Listing 3: Checking if Target Point is Behind the Car

The speed controller operates based on the difference between the desired linear velocity, received from the `/velocity` topic, and the current velocity obtained from the vehicle's twist message. This speed error is fed into a PD control law, where the proportional and derivative terms are scaled using manually tuned gains (`Kp_v`, `Ki_v` and `Kd_v`) to generate throttle commands. Notably, braking is not actively applied when the car overshoots the target speed; instead, throttle is simply reduced to zero. This decision likely aims to avoid abrupt deceleration in simulation and ensure a more natural, smooth driving response.

```
double target_speed = target_velocity_.twist.linear.x;
double current_speed = current_twist_.twist.linear.x;
double speed_error = target_speed - current_speed;

speed_integral_ += speed_error * dt;
double speed_derivative = (speed_error - prev_speed_error_) / dt;
prev_speed_error_ = speed_error;

double Kp_v = 38.0, Ki_v = 0.0, Kd_v = 0.2;

if (speed_error >= 0) {
    // when car speed slower than target speed : throttle
    cmd.Throttle = Kp_v * speed_error + Ki_v * speed_integral_ + Kd_v * speed_derivative;
    cmd.Throttle = std::clamp(cmd.Throttle, 0.0f, 0.6f);
    cmd.Brake = 0.0;
} else {
    // when car speed faster than target speed : brake
    cmd.Throttle = 0.0;
    cmd.Brake = 0.0;
}
```

Listing 4: Speed Control Using PID

The steering controller computes the angular deviation between the current vehicle orientation and the desired heading toward the target point. Using `atan2(dy, dx)`, it determines the desired yaw and subtracts the actual yaw to calculate the steering error. This error is wrapped within the range $[-\pi, \pi]$ for continuity and is fed into a full PID control formula, which includes proportional, integral, and derivative components. The resulting steering command is then clamped to within ± 1.0, respecting actuator limits. This ensures stable lateral control, allowing the vehicle to track the path accurately and avoid oscillatory behavior.

```
double desired_yaw = std::atan2(dy, dx);
double steer_error = current_yaw - desired_yaw; // negative: turn left, positive: turn right

while (steer_error > M_PI) steer_error -= 2 * M_PI;
while (steer_error < -M_PI) steer_error += 2 * M_PI;

// insert test output
ROS_INFO_STREAM("current_yaw:" << current_yaw
                << ",desired_yaw:" << desired_yaw
                << ",steer_error:" << steer_error);

steer_integral_ += steer_error * dt;
double steer_derivative = (steer_error - prev_steer_error_) / dt;
```

```
prev_steer_error_ = steer_error;

double Kp_s = 1.0, Ki_s = 0.1, Kd_s = 0.05;
cmd.Steering = Kp_s * steer_error + Ki_s * steer_integral_ + Kd_s * steer_derivative;
cmd.Steering = std::clamp(cmd.Steering, -1.0f, 1.0f);
```

Listing 5: Steering Control Using PID

The `tf` library is utilized to interpret the vehicle's current orientation in terms of yaw angle. Specifically, `tf::getYaw()` extracts the yaw from the quaternion representing the car's pose orientation, enabling the controller to compute heading alignment with the target point. This yaw information is critical for determining the angular steering error, which is then passed through a PID controller to generate smooth steering commands. By leveraging `tf`, the node simplifies the transformation from quaternion-based orientation to usable Euler angles for real-time vehicle control.

Overall, dummy_controller_node serves as a modular and interpretable control unit suitable for testing fundamental autonomous driving logic within ROS, offering a balance between simplicity and effectiveness for academic or prototyping purposes.

5 Results

Through the integration of ROS and Unity via the ROS–Unity bridge, our system successfully enables the autonomous vehicle to navigate from a predefined start point to the goal within the simulated urban environment. The following key results were achieved:

- **Traffic Light Interaction**: The vehicle encountered and correctly responded to five traffic light events, performing automatic stopping and restarting without human intervention.
- **NPC Vehicle Interaction**: Two interactions with NPC vehicles were handled successfully, with the ego vehicle adjusting its speed to follow without collisions.
- **Collision and Lane Adherence**: Throughout the entire route, the vehicle avoided collisions with static and dynamic obstacles and remained within lane boundaries.
- **Performance**: The best recorded run completed the mission in 3 minutes and 33 seconds, including delays due to red lights, while the overall system achieved a success rate of approximately 75%.

These results demonstrate the effectiveness of the perception, planning, and control components implemented in the project.

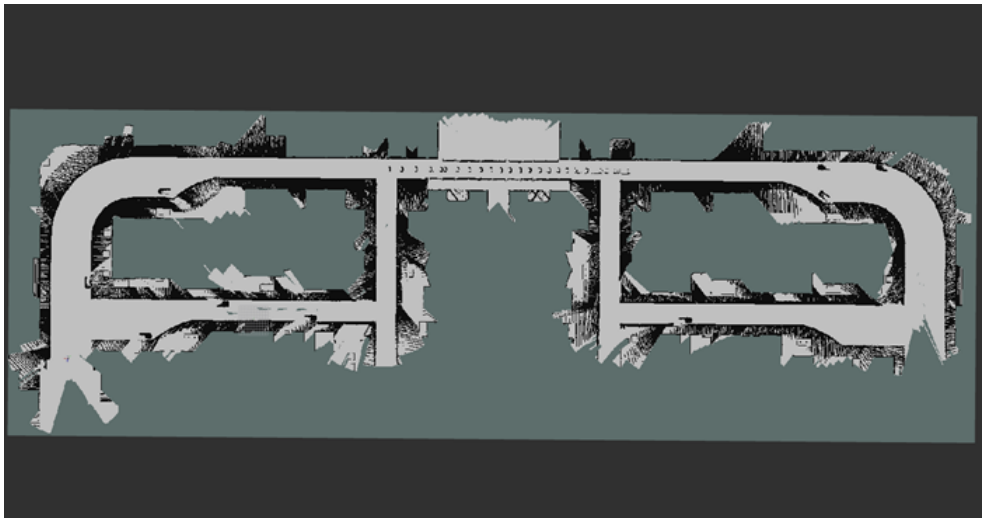

Figure 5: Full projected 2D map in RViz

6 Contributions

- Gaojian Xie: semantic camera, short-term goal generation, path planner, documentation.
- Haowen Shi: perception pipeline, path planner, documentation.
- Jia Cao: trajectory planner, vehicle control module, documentation.
- Lingheng Kong: vehicle controller, decision making, documentation.
- Zhuoya Zhang: vehicle controller, state machine, decision making, documentation.

7 Challenges

1. **Dynamic obstacles in OctoMap:**
   By default, OctoMap can only perceive static obstacles; dynamic obstacle shadows persist and interfere with path planning.
   Solution: Ignore the dynamic shadow region within 10 meters in front of the vehicle in OctoMap to avoid false obstacle blocking during path planning.

2. **Unreachable depth at second traffic light:**
   The second traffic light in the Unity map cannot be read by the depth camera for some reason, making it impossible to transform the detected position from the semantic camera to the RGB camera frame using `tf2`.
   Solution: Simulate real depth using dynamic pseudo-depth values.

3. **Node launch order dependency:**
   The `short_term_node` and `path_planner_node` must be launched in a specific order; the short-term planner must wait until the path planner is ready before publishing the first short-term goal.
   Solution: Use `ros::topic::waitForMessage<std_msgs::Bool>` to wait for readiness, though this sometimes fails and the issue is not fully resolved.

4. **Traffic light selection ambiguity:**
   Determining which detected traffic light is relevant to the ego vehicle can be ambiguous.
   Solution:
   (a) Choose the traffic light closest to the center of the semantic image.
   (b) Choose the traffic light closest to the center of the semantic camera in the X-axis direction.
   (c) Ignore cases where only one traffic light is detected, to avoid repeated recognition.

5. **Handling trajectory lag in tracking:**
   A typical problem in trajectory tracking occurs when the reference path lags behind the vehicle, often due to higher-than-expected velocity or sparsely spaced waypoints. To prevent the controller from targeting outdated points, the system combines nearest-point matching with forward-only index updates. The closest point is searched within a forward window and the index advances only if it leads the vehicle. A time-based update further ensures temporal consistency, while a lookahead distance selects a forward target pose for predictive guidance. This ensures stable, smooth, and forward-consistent tracking.

6. **Success rate and turning failure Case:**
   The observed success rate is approximately 75%. A typical failure occurs when the vehicle is preparing to turn and the traffic light suddenly changes from green to red. After braking and attempting to restart, the vehicle fails to complete the turn due to insufficient speed and suboptimal PID control parameters. This suggests that improved low-speed control tuning and transition handling may be necessary to ensure consistent behavior in dynamic intersections.